Describing a Table

Reading a delimited file into one array per column, NaN and the tests for it, counting the categories in a column, mean, median, standard deviation and quantiles, pairwise against listwise deletion, grouped summaries and bar charts, histograms and their bin edges, the shapes a distribution takes, scatter plots, correlation and its two limits, joint histograms, the classes of a label compared on a grid of plots, and the encoding that turns the table into the matrix of floats a neural network takes as input.

01 Reading a delimited file
02 Missing values and NaN
03 Counting rows and categories
04 Summary statistics
05 Pairwise and listwise deletion
06 Grouped summaries
07 Bar charts
08 Histograms
09 Bin count and range
10 Distribution shapes
11 Scatter plots
12 Correlation
13 Joint histograms
14 Comparing groups
15 Encoding a table for a neural network
16 Loop forms and array equivalents

Scope. Arrays, slicing, masks and reductions are assumed. A neural network takes as input a matrix of floats: one row per example, the same columns in every row, and a number in every cell. A table exported from a database is not one. Its columns hold different kinds of value, some of them text, and not every field is filled.

After reading it you should be able to do the following without looking anything up.

- Read a delimited file into one array per column, each with a dtype of its own, or into one array for the whole table with the categories coded as integers.
- Represent a missing number, and say what a sum, a mean or the weighted sum inside a neural network returns when it contains one.
- Count how many rows fall into each category of a categorical column, including a category with no rows.
- Compute mean, median, standard deviation, extremes and quantiles, and say which of them a single extreme value moves.
- Reduce every column of a table in one call with `axis=0`, and sort a column that has gaps in it without the NaNs ending up where the largest values should be.
- Skip missing values pairwise rather than dropping whole rows, and state the difference it makes.
- Summarise a numeric column within each category of a categorical one, and draw counts per category as a bar chart.
- Build a histogram with a stated bin count and range, and say what each returned array holds.
- Say how the bin count and the range change the picture, and choose both deliberately.
- Plot two columns against each other and describe the relationship in words.
- Compute a Pearson correlation coefficient and state the two things it does not measure.
- Bin two columns jointly, display the result, and get the orientation right.
- Count the classes of a label, state the accuracy of always predicting the largest one, and compare a feature across the classes on a grid of plots.
- Encode the table as the input of a neural network, with categories one-hot encoded, gaps filled with the median and marked in indicator columns, and numeric columns standardised, and say why each step is needed.

Every example assumes `import csv`, `import io`, `import numpy as np` and `import matplotlib.pyplot as plt`. The running example is an export of 9000 customer records with eight columns: four numeric (`age`, `income`, `tenure_months`, `score`), three categorical (`region`, `plan`, `channel`), and the label `churn`, 1 for a customer who left and 0 for one who stayed. `score` runs from 0 to 100 by definition, `tenure_months` from 0 to 180, and `income` is a positive amount with no fixed upper bound. Values in `# ->` comments are rounded to the digits shown.

01

Reading a delimited file

Every field in a CSV file is text. Nothing in the file records what type a column is or what encoding it was written in, and the rules for splitting a line into fields are a convention rather than a standard.

```
age,income,tenure_months,score,region,plan,channel,churn
38,22207.56,47,59.5,North,standard,online,0
,47425.23,36,50.6,Eastfield,premium,shop,0
```

```
open("register.csv", newline="", encoding="utf-8").readline()
# -> 'age,income,tenure_months,score,region,plan,channel,churn\r\n'
# without newline="" the \r\n is translated to \n before it is returned
```

The columns do not share a type: three of them are text and five are numbers. The third row has nothing before its first comma, which is what a missing value looks like in a CSV export.

CSV CONVENTIONS AND RFC 4180

There is no CSV standard that every writer follows. RFC 4180 describes the common form, and most tools approximate it:

- Fields are separated by a delimiter. Usually a comma, but a semicolon where the comma is the decimal separator, and a tab in a file called TSV.
- A field containing the delimiter, a quotation mark or a newline is wrapped in quotation marks.
- A quotation mark inside a quoted field is written twice.
- The first line is a header by convention. Nothing in the file marks it as one.

None of that is enforced. A file can change its column count halfway down, quote inconsistently, or have a header that does not match what follows, and still be a CSV.

Splitting a line on the delimiter does not take the quoting into account, so it also splits the quoted fields that contain one:

```
line = 'Smith, John,42,"Oxford, Oxfordshire"'

line.split(",")          # -> ['Smith', ' John', '42', '"Oxford', ' Oxfordshire"']   5 fields
next(csv.reader([line])) # -> ['Smith', ' John', '42', 'Oxford, Oxfordshire']        4 fields
```

The other two rules have the same consequence. A doubled quotation mark is one character, not two, and a newline inside a quoted field does not end the record, so a file of 9000 records can have more than 9001 lines in it.

```
doubled = 'id,note\n7,"she said ""no"" twice"\n'
list(csv.reader(io.StringIO(doubled)))[1]    # -> ['7', 'she said "no" twice']

wrapped = 'id,note\n7,"line one\nline two"\n'
len(list(csv.reader(io.StringIO(wrapped))))  # -> 2 records
len(wrapped.strip().split("\n"))             # -> 3 lines
```

The csv module implements all three rules, which is why a CSV is read through it rather than through `str.split`.

TYPES, ENCODING AND MISSING-VALUE MARKERS

Three things have to come from outside the file, and a wrong assumption about any of them raises no error.

- **The types.** Every field is text. That `38` is an age and `59.5` a score is a decision made by the reader.
- **The encoding.** Nothing in a CSV records whether it is UTF-8 or something older, so a name with an accent in it is either decoded correctly or turns into mojibake, depending on what the reader assumed.
- **What counts as missing.** An empty field is one convention. `NA`, `NULL`, `-999` and `-1` are others, and code that does not check for them loads a sentinel as a number and averages it.

MIXED COLUMN TYPES

`np.loadtxt` and `np.genfromtxt` read a delimited file straight into an array. They fit a file that is numeric throughout. This one is not: a single array has a single dtype, so a table with text and numbers in it has to become either several arrays or one array of strings, and an array of strings cannot be averaged.

The rows are therefore read as text first, and each column is converted separately. `csv.DictReader` yields one dictionary per row, keyed by the header line, so columns are named rather than numbered.

```
with open("register.csv", newline="", encoding="utf-8") as f:
    records = list(csv.DictReader(f))

len(records)            # -> 9000
records[0]["age"]       # -> '38'      a string, not an int
records[1]["income"]    # -> '47425.23' also a string
records[1]["age"]       # -> ''        the empty field from the line above
```

The csv module does its own line-ending handling, so its documentation specifies `newline=""` for any file passed to it. This file ends its lines with `\r\n`, which a plain text-mode read translates to `\n` before it reaches the csv reader. On a file like this one the parsed rows are identical either way; the difference shows in a quoted field that contains a newline of its own, and on writing, where the translation can add a stray `\r`. `encoding="utf-8"` is stated explicitly, because the default encoding of `open` depends on the machine.

ONE ARRAY PER COLUMN

Each column then becomes its own array with its own dtype. The numeric ones become `float64`, the categorical ones stay as strings.

```
region = np.array([r["region"] for r in records])    # (9000,) dtype '<U9', text
tenure = np.array([float(r["tenure_months"]) for r in records])   # (9000,) float64

tenure.dtype            # -> dtype('float64')
region.dtype            # -> dtype('<U9')   unicode, up to 9 characters
region[:3]              # -> ['North' 'Eastfield' ...]
```

A string array has a fixed width per element, chosen from the longest string present at creation. Assigning a longer string into it later truncates the string without a warning, so string arrays are built from the complete data rather than grown.

The same conversion on `income` fails on the empty fields:

```
np.array([float(r["income"]) for r in records])
# ValueError: could not convert string to float: ''
```

ONE ARRAY FOR THE WHOLE TABLE

The alternative to one array per column is to make the text columns numeric and keep the whole table in a single array: each category becomes an integer code, and the result is one two-dimensional block of floats, one row per customer and one column per variable.

```
NUMERIC = ["age", "income", "tenure_months", "score"]
CATEGORICAL = ["region", "plan", "channel"]
CATEGORIES = {"region": ["North", "Harbour", "Midland", "Eastfield", "Southgate", "Upland"],
              "plan": ["basic", "standard", "premium", "legacy"],
              "channel": ["shop", "phone", "online"]}
COLUMNS = NUMERIC + CATEGORICAL

CODES = {c: {name: i for i, name in enumerate(CATEGORIES[c])} for c in CATEGORICAL}

rows = []
for r in records:
    numeric = [float(r[c]) if r[c] != "" else np.nan for c in NUMERIC]
    coded = [float(CODES[c][r[c]]) for c in CATEGORICAL]
    rows.append(numeric + coded)

table = np.array(rows, dtype=np.float32)
table.shape             # -> (9000, 7)
table[15]               # -> [52.  nan  23.  65.8  4.  2.  0.]   no income
```

Every per-column statistic then becomes one call with `axis=0`, and the table already has the shape a neural network takes as input: a matrix with one row per example and one column per **feature**, the name for an input variable.

The cost is that the codes are labels, not quantities. Southgate is 4 and Midland is 2, which does not make Southgate twice Midland, and nothing in the array records that difference.

```
np.nanmean(table, axis=0)[4:]    # -> [2.17 1.14 1.13]   means of the codes, without meaning
```

Any operation that reads the column as a quantity, a mean or a distance or a linear model, treats the categories as having an order they do not have. The repair is one column per category instead of one code, which is called one-hot encoding. Both representations are used: one array per column keeps each column's own dtype, so a category cannot be averaged by accident, and one array for the table allows reductions with `axis=0` and has the shape a model takes.

PANDAS

The library `pandas` does all of the above in one call: `pd.read_csv("register.csv")` infers a type per column, keeps the names, and represents empty fields as missing automatically. The type of each column and what counts as missing both change the statistics computed from the table, and reading the file by hand makes both explicit.

02

Missing values and NaN

An empty field is not a zero and not an empty string once it reaches a numeric column. It needs a value that arithmetic propagates rather than ignores.

Floating point reserves one: **NaN**, not-a-number, written `np.nan`. It is a float, so it sits in a float array without changing the dtype, and every operation that touches it produces NaN again.

```
def column(name):
    """One numeric column as floats, with empty fields as NaN."""
    return np.array([float(r[name]) if r[name] != "" else np.nan for r in records])

income = column("income")
income.dtype            # -> dtype('float64')  NaN is a float
income[15]              # -> nan   row 15 has no income
```

NAN PROPAGATION

```
np.nan + 1              # -> nan
income.sum()            # -> nan     one missing value makes the total NaN
income.mean()           # -> nan
income.max()            # -> nan
```

Every reduction over a column with a gap returns NaN until the gaps are handled explicitly, so a gap cannot change a result unnoticed.

NAN IN A WEIGHTED SUM

```
x = np.array([52.0, np.nan, 23.0, 65.8])     # the four numbers of row 15
w = np.array([0.02, 0.0, -0.01, 0.03])       # one weight per input

x * w                   # -> [ 1.04    nan -0.23   1.974]
x @ w                   # -> nan
np.nan * 0.0            # -> nan
```

A neural network multiplies each of its inputs by a weight and adds the products, and one NaN among the inputs makes that weighted sum NaN. A weight of zero does not prevent it, because NaN times zero is NaN. Training a network averages an error measure, the loss, over many rows and changes every weight by an amount computed from it, so a single row with a gap makes the loss NaN and writes NaN into every weight. From then on the network returns NaN for every input, complete rows included. Every gap has to be filled, or its row removed, before the table is used to train a network.

NAN COMPARISONS AND NP.ISNAN

NaN is not equal to itself, which follows from the floating-point standard rather than from NumPy. A comparison against NaN is false, including a comparison with another NaN, so `== np.nan` never finds anything.

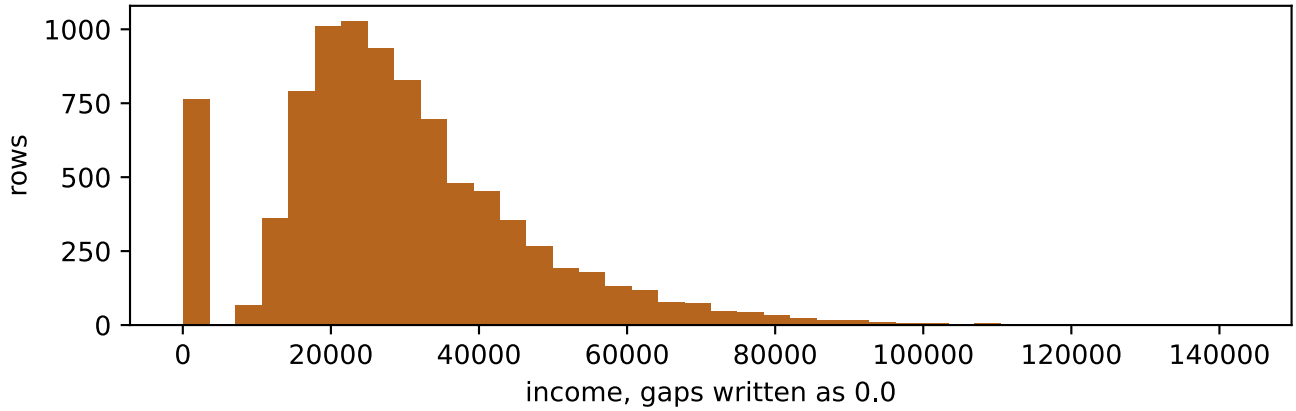
```
np.nan == np.nan        # -> False
income == np.nan        # -> all False, a mask that selects nothing

np.isnan(income)        # (9000,) -> (9000,) bool, True where the value is missing
np.isnan(income).sum()  # -> 763    how many are missing
(~np.isnan(income)).sum()   # -> 8237  how many are present
```

`np.isnan` is the test for NaN. It takes a float array and gives a boolean array, which is an ordinary mask from there on.

ALTERNATIVES TO NAN

Two alternatives to NaN change the data. Writing `0.0` for a missing income adds 763 customers who earn nothing, which pulls the mean down and creates a spike at zero in every histogram. Dropping every row that has a gap in any column also discards the values that were present in those rows.



Income with the 763 gaps written as 0.0. The bar at the left edge is the gaps; no recorded income is below 6138. The mean falls from 31838 to 29139.

03

Counting rows and categories

Two counts come before any statistic: how many rows there are, and how they are spread across the categories of each categorical column.

```
plan = np.array([r["plan"] for r in records])      # (9000,) text

(plan == "premium").sum()           # -> 1828   one category, counted with a mask
np.unique(plan)                     # -> ['basic' 'legacy' 'premium' 'standard']
names, counts = np.unique(plan, return_counts=True)
list(zip(names, counts))           # -> [('basic', 3103), ('legacy', 1250), ...]
```

`np.unique` returns the categories in sorted order, which is alphabetical for text. That order comes from the spelling of the categories, not from the data.

CATEGORIES WITH NO ROWS

`np.unique` reports only the categories that occur. If a category is defined by the domain and happens to be absent, it is missing from the output, and a result indexed by position shifts without an error. Counting against a fixed list of categories avoids it and makes the zero visible.

```
counts = {name: int((plan == name).sum()) for name in CATEGORIES["plan"]}
counts                # -> {'basic': 3103, 'standard': 2819, 'premium': 1828, 'legacy': 1250}
sum(counts.values())  # -> 9000, every row accounted for
```

Checking that the counts add up to the number of rows catches a misspelled category, a trailing space in the file, and a category that was not in the list.

MISSING VALUES PER COLUMN

The same count, applied to the gaps: one count per numeric column, including the column with no gaps.

```
for name in ["age", "income", "tenure_months", "score"]:
    v = column(name)
    print(name, int(np.isnan(v).sum()))

# age            564
# income         763
# tenure_months    0
# score          373
```

04

Summary statistics

Five statistics summarise a numeric column: the mean and the median give its centre, the standard deviation its spread, and the minimum and maximum its extremes. A single extreme value affects them differently.

```
score = column("score")
s = score[~np.isnan(score)]         # (9000,) -> (8627,) the present values

s.mean()                # -> 58.85   the balance point
np.median(s)            # -> 59.10   the middle value
s.std()                 # -> 13.16   root mean square distance from the mean
s.min(), s.max()        # -> 8.8, 100.0
np.quantile(s, [0.25, 0.5, 0.75])   # -> [50.1 59.1 67.6]  the quartiles
```

REDUCTIONS WITH AXIS=0

With the table in one array, a reduction takes `axis=0` and returns one value per column. The axis named is the one that disappears, so a `(9000, 7)` table reduces to `(7,)`.

```
table.mean(axis=0)              # (9000, 7) -> (7,)  [nan nan 26.73 nan 2.17 1.14 1.13]
np.nanmean(table, axis=0)[:4]   # -> [   43.12 31837.70    26.73    58.85]
np.nanmin(table, axis=0)[:4]    # -> [   18.    6138.67     0.       8.8 ]
np.nanmax(table, axis=0)[:4]    # -> [   84.  142610.66   145.     100.  ]
```

The gaps still have to be skipped, and now on every column in one call: three of the four numeric columns have a NaN in them, so the plain `mean` returns NaN in three of the first four positions. Pairing the result with the column names labels each value.

```
dict(zip(COLUMNS, map(float, np.nanmean(table, axis=0))))
```

`map(float, ...)` is there because NumPy prints its own scalar types in full, as `np.float32(43.12)` rather than `43.12`.

Counting works the same way, and `axis` sets what is being counted.

```
missing = np.isnan(table)                  # (9000, 7) bool

np.count_nonzero(missing, axis=0)          # -> [564 763   0 373   0   0   0]  per column
np.count_nonzero(np.any(missing, axis=1))  # -> 1597   rows with a gap anywhere
np.where(np.any(missing, axis=0))[0]       # -> [0 1 3]  which columns have gaps
```

The middle line counts the rows with a gap somewhere, 1597 of 9000, so dropping every incomplete row keeps only 7403.

SORTING A COLUMN WITH GAPS

`np.sort` returns a sorted copy and `np.argsort` returns the order rather than the values. On a column with gaps both put the NaNs at the end, so the first positions hold the smallest values and the last positions hold NaN rather than the largest values.

```
inc = table[:, 1]               # income, 763 of the 9000 missing

np.sort(inc)[:3]                # -> [6138.67 7221.34 7492.29]   the three smallest
np.sort(inc)[-3:]               # -> [nan nan nan]               not the three largest
```

The gaps can be dropped before sorting, or moved to the front:

```
present = inc[~np.isnan(inc)]            # (9000,) -> (8237,)
np.sort(present)[-3:]                    # -> [132465.06 133638.58 142610.66]

filled = inc.copy()
filled[np.isnan(filled)] = -np.inf       # NaN sorts last, -inf sorts first
np.sort(filled)[-3:]                     # -> [132465.06 133638.58 142610.66]
```

`-np.inf` is negative infinity, a float that compares below every real number, so the gaps sort to the front instead of the back.

Dropping is shorter; moving keeps the array's length, which is needed when the positions matter rather than the values.

`np.argsort` returns row numbers, so one column can be ordered and another read off it. Dropping the gaps breaks that, because every index after a dropped row shifts.

```
order = np.argsort(inc)                  # (9000,) row numbers, poorest first
inc[order[-3:]]                          # -> [nan nan nan]   the NaNs sort last here too

n_missing = int(np.isnan(inc).sum())     # 763
top3 = order[-n_missing - 3 : -n_missing]
inc[top3]                                # -> [132465.06 133638.58 142610.66]
table[top3, 0]                           # -> [59. 67. 61.]   the ages of those three

# or keep the original row numbers through the filtered sort
np.flatnonzero(~np.isnan(inc))[np.argsort(present)][-3:]    # the same three rows
```

`np.sort` also takes an axis, and on a table it sorts every column independently. The result is useful for reading off the extremes of each column, and its rows are no longer customers: the smallest age and the smallest income almost certainly belong to two different people.

```
np.sort(table, axis=0)[:3]       # (9000, 7) -> (3, 7)  the three smallest of each column
```

QUANTILES

`np.quantile(s, q)` returns the value below which the fraction `q` of the data falls. The three at 0.25, 0.5 and 0.75 are the **quartiles**: a quarter of the scores sit below 50.1, half below 59.1, and three quarters below 67.6. The middle one is the median by definition. When the fraction does not land exactly on an observation NumPy interpolates between the two either side of it, which is why a quartile is usually not a value that appears in the data. `np.percentile` is the same function on a 0 to 100 scale.

The distance between the outer two is the **interquartile range**, 17.5 here. It describes the spread of the middle half of the data. Because it is computed from two quartiles, one extreme value leaves it at 17.5, while the standard deviation grows from 13.2 to 107.8.

MEAN AND MEDIAN

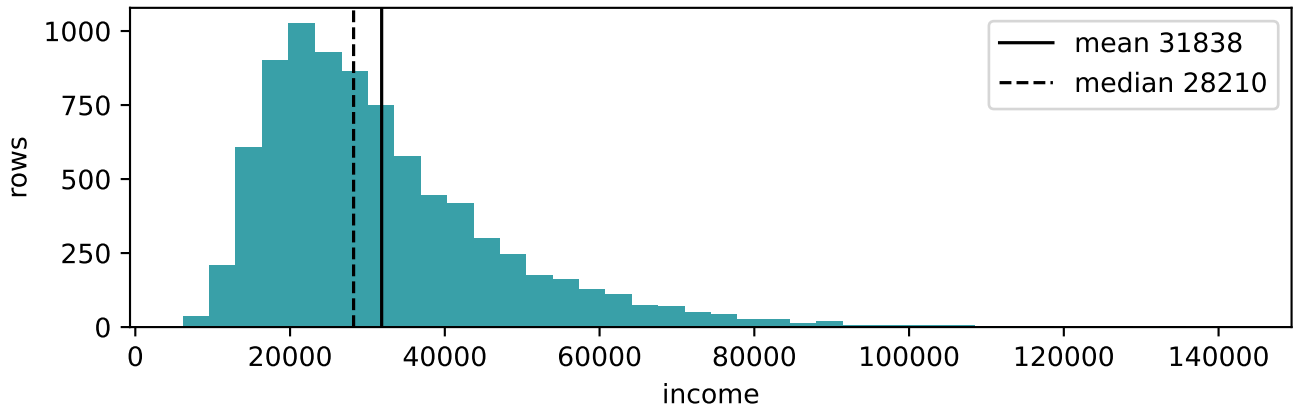
The mean and the median measure the centre in different ways, and the gap between them describes the shape of the column. On a symmetric column they nearly agree, as `score` does above. On a skewed one they do not:

```
inc = income[~np.isnan(income)]     # (9000,) -> (8237,)

inc.mean()              # -> 31837.68
np.median(inc)          # -> 28210.26   about 3600 lower
```

Income has a long right tail: most values sit low and a few sit far above. The mean is pulled towards the tail because it weighs every value by its size; the median is not, because it only counts how many values lie on

each side. A mean above the median indicates a long right tail, which the mean on its own does not show.



The 8237 recorded incomes. The mean, 31838, sits to the right of the median, 28210, because of the long right tail.

DDOF

`np.std` divides by `n`. Many statistics texts and `statistics.stdev` divide by `n - 1`, which corrects the bias that arises when the data is a sample from a larger population: the deviations are measured from the sample mean, which sits exactly where it makes their sum of squares as small as possible, so dividing by `n` gives a variance that is on average smaller than the population variance by a factor of `(n - 1) / n`. NumPy exposes the choice as `ddof`, the delta degrees of freedom, and defaults it to 0.

```
s.std()                # -> 13.1642   ddof=0, divides by n
s.std(ddof=1)          # -> 13.1649   divides by n - 1
```

Which one to use follows from what the rows are. The spread of *these* 8627 scores is a property of the rows in hand, so `ddof=0` computes it exactly and NumPy's default is the right one. `ddof=1` is for rows that are a sample, where the number is an estimate of the spread of a larger population that cannot be measured.

The difference between the two shrinks as the number of values grows. Here it is 13.1642 against 13.1649, in the fourth digit; at twenty values it is 2.6 per cent, and at five it is 12 per cent.

ROBUSTNESS TO ONE EXTREME VALUE

One impossible value, such as a score typed with an extra digit, moves the five statistics by very different amounts.

```
t = np.append(s, 10_000.0)          # (8627,) -> (8628,) one impossible score

t.mean() - s.mean()     # -> +1.15     the mean moves
np.median(t) - np.median(s)     # -> 0.0    the median does not
t.max() - s.max()       # -> +9900.0   the maximum is the error itself
t.std() - s.std()       # -> +94.7     13.2 becomes 107.8
```

The median and the quartiles are **robust**: they depend on the order of the values, not their size, so a single extreme value moves them by at most one position. The mean, the standard deviation and the extremes are not. An impossible value usually shows up in `min` or `max`, so printing those is a quick check on a new column.

05

Pairwise and listwise deletion

Leaving a missing value out of a calculation can mean dropping the whole row or dropping only that one value. The two give different numbers.

Listwise deletion, also called complete-case analysis, drops every row that has a gap in any column and computes everything on what is left. **Pairwise deletion**, also called available-case analysis, keeps for each statistic every row where the columns that statistic needs are present.

```
age, inc_c = column("age"), column("income")
score_c, ten = column("score"), column("tenure_months")

complete = ~np.isnan(age) & ~np.isnan(inc_c) & ~np.isnan(score_c) & ~np.isnan(ten)
complete.sum()                      # -> 7403   of 9000 rows remain after listwise deletion

score_c[~np.isnan(score_c)].mean()  # -> 58.8494   pairwise: 8627 values
score_c[complete].mean()            # -> 58.8511   listwise: 7403 values
```

Pairwise deletion

`score_c[~np.isnan(score_c)]`

	age	income	tenure_months	score
801	28	nan	20	54.9
802	41	20425.71	10	53.2
803	42	22549.19	57	63.1
804	41	15142.54	18	43.9
805	56	33261.76	56	68.8
806	38	14575.14	7	54.9
807	46	38143.54	50	nan
808	45	nan	25	75.3

mean of the 7 scores used: 59.16

Listwise deletion

`score_c[complete]`

	age	income	tenure_months	score
801	28	nan	20	54.9
802	41	20425.71	10	53.2
803	42	22549.19	57	63.1
804	41	15142.54	18	43.9
805	56	33261.76	56	68.8
806	38	14575.14	7	54.9
807	46	38143.54	50	nan
808	45	nan	25	75.3

mean of the 5 scores used: 56.78

Rows 801 to 808 of the register, gaps hatched. The mean of score uses the blue cells: pairwise deletion leaves out only row 807, whose score is missing, and listwise deletion also drops rows 801 and 808 for a gap in another column, with the scores they had (orange).

The first is the mean of every score that is present. The second is the mean score among customers whose record is complete in all four columns, which is a different population from the customers with a recorded score. It also discards 1224 scores that were present, because something unrelated was missing in the same row.

The loss grows with the number of columns: four columns with 6, 8, 0 and 4 per cent missing lose 18 per cent of the rows between them. In this register the gaps in different columns are unrelated, so the rows lost are a random sample. Where gaps are related, for instance when customers who withhold their income also withhold their age, listwise deletion removes a particular kind of customer from every statistic.

FUNCTIONS THAT IGNORE NANS

NumPy has a parallel set of reductions that skip NaN instead of propagating it, which is pairwise deletion in one call.

PROPAGATES NAN	SKIPS NAN	RESULT
`v.sum()`	`np.nansum(v)`	Total of the present values
`v.mean()`	`np.nanmean(v)`	Mean of the present values
`np.median(v)`	`np.nanmedian(v)`	Median of the present values
`v.std()`	`np.nanstd(v)`	Spread of the present values
`v.min()`, `v.max()`	`np.nanmin(v)`, `np.nanmax(v)`	Extremes of the present values

```
np.nanmean(score_c)     # -> 58.8494   same as masking by hand
np.nanmean(score_c) == score_c[~np.isnan(score_c)].mean()   # -> True
```

`np.nanmean` of a column that is entirely missing returns NaN and raises a `RuntimeWarning`, rather than raising an error. There is no NaN-skipping count function; the count of present values is `(~np.isnan(v)).sum()`, which is also the count `np.nanmean` divides by.

06

Grouped summaries

A grouped summary computes a statistic once for each category of another column, such as the mean score in each region.

Over the whole column the mean score is `58.85`; region by region the six means run from 51 to 67.

Each of those six is an ordinary mean over a subset of the rows, and a boolean mask selects the subset. `region == "Upland"` is True on the rows from Upland; `~np.isnan(score_c)` is True on the rows that have a score; `&` keeps the rows where both hold. Indexing the score column with that mask gives the scores to average.

```
region = np.array([r["region"] for r in records])
score_c = column("score")
REGIONS = ["North", "Harbour", "Midland", "Eastfield", "Southgate", "Upland"]

for name in REGIONS:
    sel = (region == name) & ~np.isnan(score_c)
    print(f"{name:<10} n={sel.sum():>5}  mean score {score_c[sel].mean():6.2f}")

# North      n= 1833  mean score  51.28
# Harbour    n= 1578  mean score  60.76
# Midland    n= 1738  mean score  58.55
# Eastfield  n= 1200  mean score  65.20
# Southgate  n= 1347  mean score  55.65
# Upland     n=  931  mean score  67.49
```

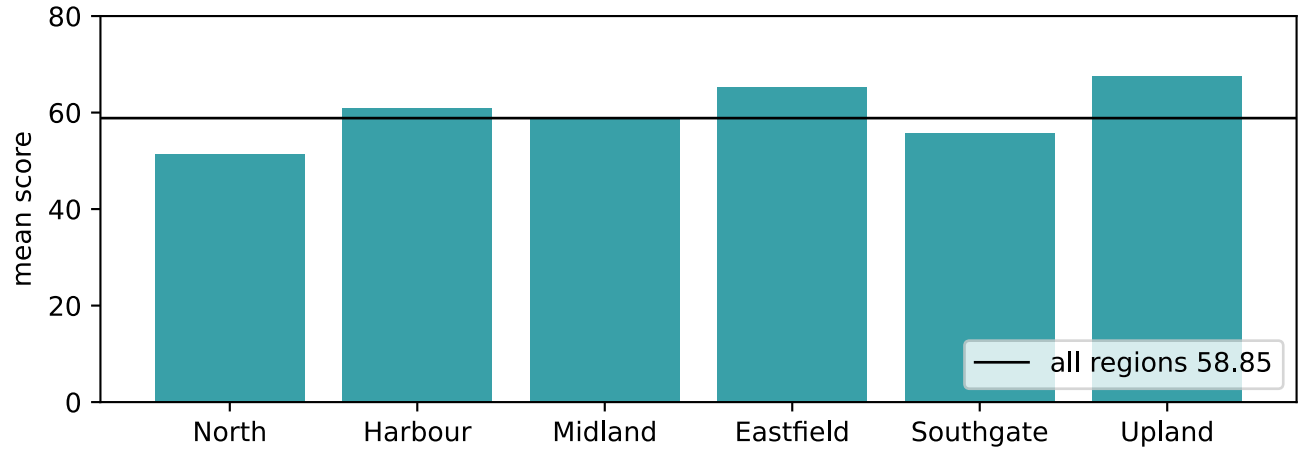
```
sel = (region == name) & ~np.isnan(score_c)
```

	region	score
801	Eastfield	54.9
802	Southgate	53.2
803	Harbour	63.1
804	North	43.9
805	Midland	68.8
806	Southgate	54.9
807	Southgate	nan
808	Upland	75.3

name	North	Harbour	Midland	Eastfield	Southgate	Upland
801	False	False	False	True	False	False
802	False	False	False	False	True	False
803	False	True	False	False	False	False
804	True	False	False	False	False	False
805	False	False	True	False	False	False
806	False	False	False	False	True	False
807	False	False	False	False	False	False
808	False	False	False	False	False	True
over all 9000 rows: `sel.sum()`	1833	1578	1738	1200	1347	931
`score_c[sel].mean()`	51.28	60.76	58.55	65.20	55.65	67.49

Rows 801 to 808 and the mask the loop builds for each region: True where the row is in that region and has a score. Row 807 is in Southgate but has no score, so its entry is False (orange); the counts and means below are the loop's output over all 9000 rows.

The loop prints the group size next to each mean: a mean over 931 rows is less precise than one over 1833, and an empty group gives NaN.



The mean score in each region, and the mean of all 8627 scores, 58.85, as the line. The overall mean does not show the spread of the region means, from 51 to 67.

DELETION INSIDE A GROUP

The mask above tests the column being averaged. Testing completeness across all columns instead changes every group, and it changes them unevenly, because the gaps are not spread evenly across regions. As for the whole column, the gaps are skipped only in the column the statistic is about.

```
sel_pairwise = (region == "Upland") & ~np.isnan(score_c)
sel_listwise = (region == "Upland") & complete

score_c[sel_pairwise].mean()     # -> 67.49   over 931 rows
score_c[sel_listwise].mean()     # -> 67.66   over 801 rows
```

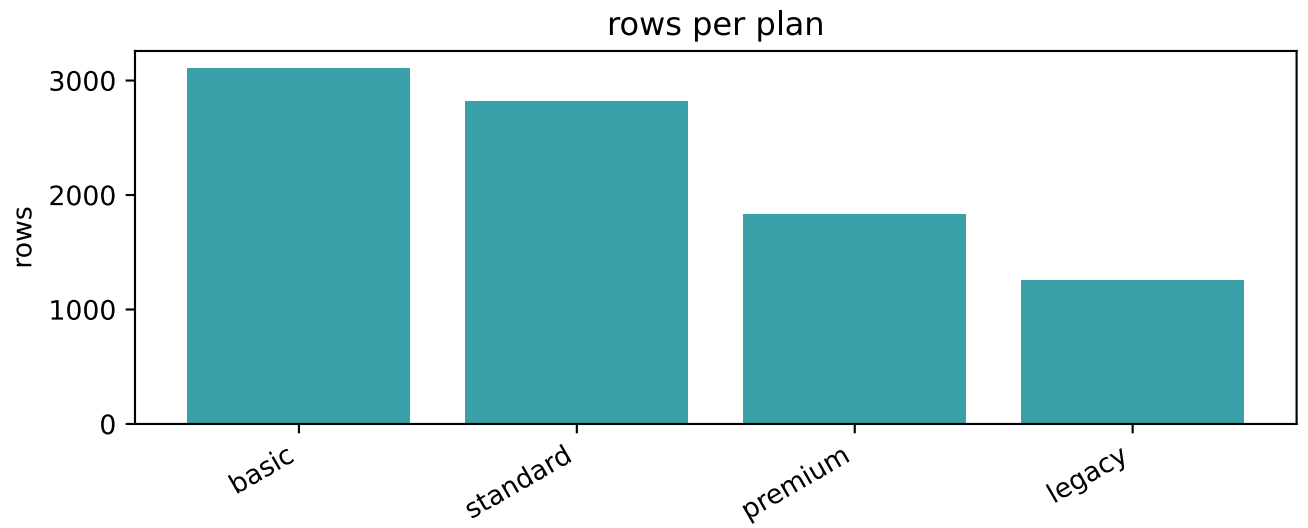
07

Bar charts

A bar chart shows one count per category as the height of a bar. `plt.bar` draws the bars, and `plt.xticks` and `plt.ylabel` label them.

```
names = CATEGORIES["plan"]
counts = [int((plan == name).sum()) for name in names]

plt.figure(figsize=(7, 3))
plt.bar(range(len(names)), counts, color="#39a0a8")
plt.xticks(range(len(names)), names, rotation=30, ha="right")
plt.ylabel("rows")
plt.title("rows per plan")
plt.tight_layout()
plt.show()
```



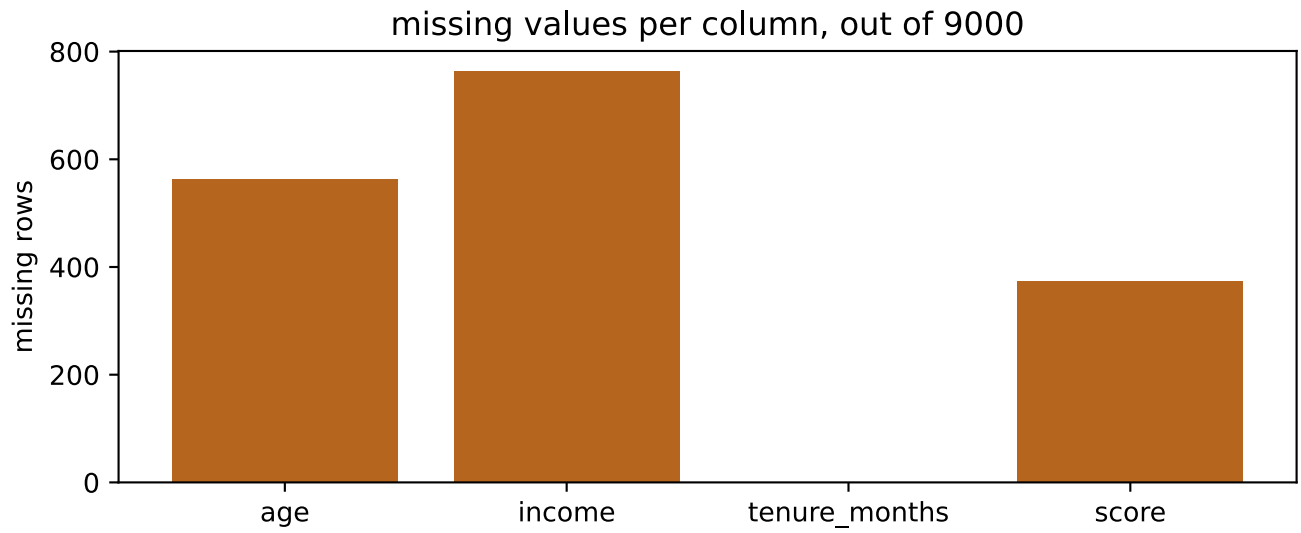
`plt.bar` places bars at the x positions it is given, so the positions are `range(len(names))` and the names are attached afterwards with `plt.xticks`. Rotating long labels and anchoring them at their right end (`ha="right"`) keeps them from overlapping. `plt.tight_layout()` resizes the axes so the rotated labels are inside the figure rather than cut off at the edge.

AXIS LABELS AND COUNTS

- **A labelled y axis.** Counts, percentages and thousands of euros all look the same without one.
- **Every category, including the empty ones.** A bar chart built from `np.unique` output omits the categories with no rows, and a missing bar is easy to overlook.
- **The number of rows the figure is drawn from.** If gaps were skipped, the figure describes the present values, and the count of those belongs in the title or the caption.

```
cols = ["age", "income", "tenure_months", "score"]
miss = [int(np.isnan(column(c)).sum()) for c in cols]

plt.figure(figsize=(7, 3))
plt.bar(range(len(cols)), miss, color="#b5651d")
plt.xticks(range(len(cols)), cols)
plt.ylabel("missing rows")
plt.title(f"missing values per column, out of {len(records)}")
plt.tight_layout()
plt.show()
```



The missing values per column, drawn by the same three calls. The column with no gaps gets a bar of height zero, which shows because the column is in the list: drawn only from the columns that have gaps, `tenure_months` would be absent and nothing would mark its absence.

A bar chart is for a categorical column, where the bars are separate things in a chosen order. A continuous column needs its values grouped into intervals first, which is a histogram.

08

Histograms

A histogram divides the value range into intervals and counts how many values fall into each. It is a reduction: thousands of numbers become one number per interval.

```
counts, edges = np.histogram(s, bins=20, range=(0, 100))

counts.shape            # -> (20,)   one count per bin
edges.shape             # -> (21,)   one more edge than bin
edges[:4]               # -> [ 0.  5. 10. 15.]   width 5.0 each
counts.sum()            # -> 8627    every value is in a bin
```

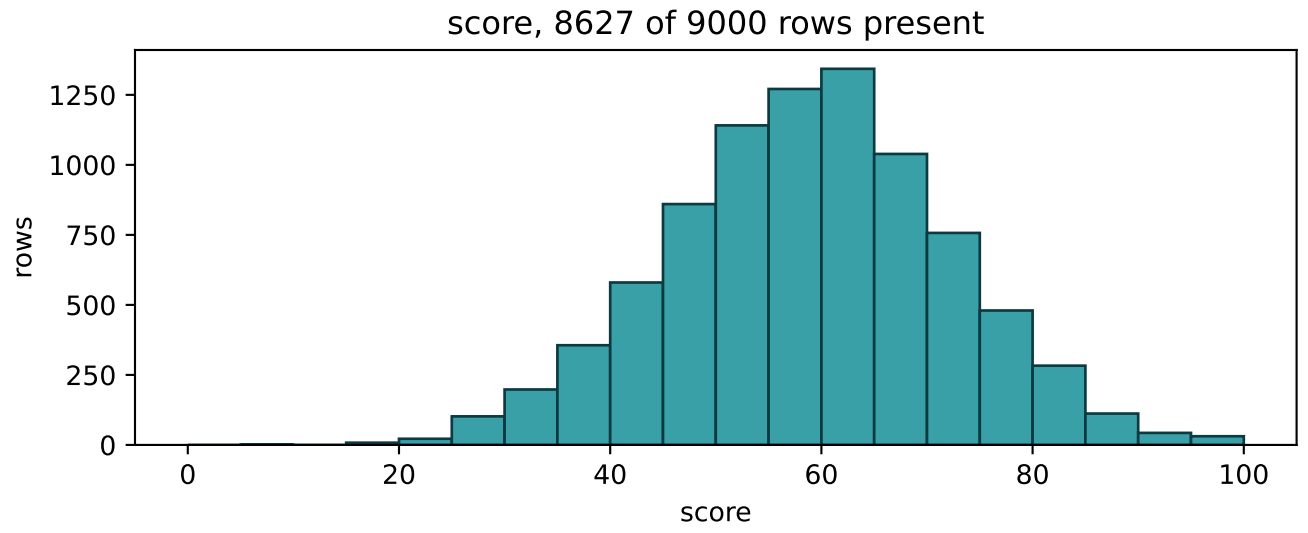
The two returned arrays differ in length by one, because `n` intervals need `n + 1` boundaries. Bin `i` covers `edges[i]` up to `edges[i+1]`, with the left edge included and the right edge excluded, except for the last bin, which includes both, so that the maximum value is counted.

If `counts.sum()` is smaller than the number of values, some values fell outside `range` and were left out without a warning.

PLT.HIST

`plt.hist` computes and draws in one call, taking the same `bins` and `range` arguments. It cannot accept NaN, so the gaps come out first.

```
plt.figure(figsize=(7, 3))
plt.hist(s, bins=20, range=(0, 100), color="#39a0a8", edgecolor="#0b3a40")
plt.xlabel("score")
plt.ylabel("rows")
plt.title(f"score, {len(s)} of 9000 rows present")
plt.tight_layout()
plt.show()
```



Nothing in a histogram shows that 373 rows were left out, so the title states the number of present values.

DENSITY=TRUE

```
c, e = np.histogram(s, bins=20, range=(0, 100), density=True)

c.sum()                 # -> 0.2       not 1
(c * np.diff(e)).sum()  # -> 1.0       the area is 1
```

`np.diff` gives the gaps between consecutive values, so `np.diff(e)` is the width of each bin and the product above is height times width, bin by bin.

`density=True` scales the counts so the total area is one, not so the values sum to one. With bins 5 wide, each returned value is a count per unit of score. It makes histograms with different bin widths comparable, and it means the numbers on the y axis are no longer rows.

09

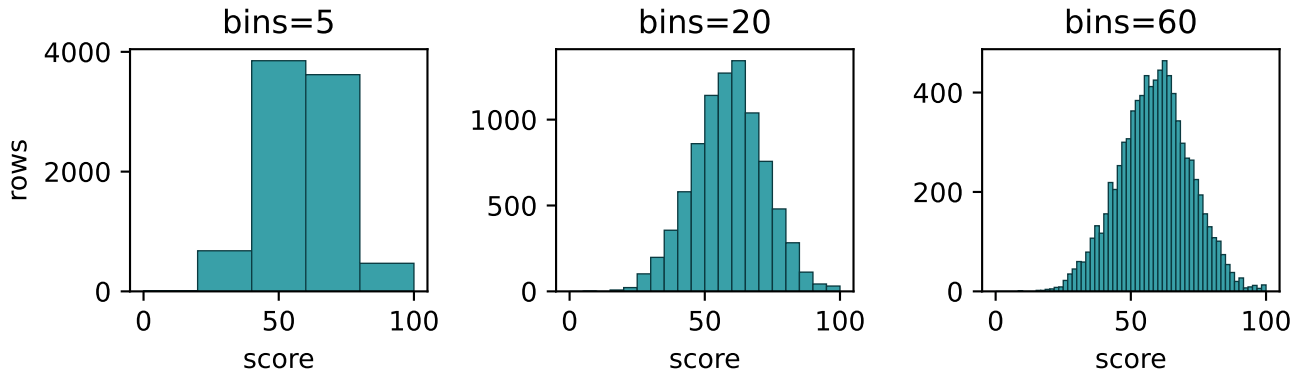
Bin count and range

The bin count and the range both change the histogram, and the data does not determine either of them.

BIN COUNT

With few bins the histogram shows only the coarsest shape; with many, each bar holds few values and neighbouring bars differ by chance. The same 8627 scores, binned three ways:

BINS	WIDTH	TALLEST BIN	EMPTY BINS	APPEARANCE
5	20.0	3852	0	That the middle is fuller than the ends, and nothing else
20	5.0	1343	2	A single rounded peak with thin tails
60	1.67	464	8	The same peak, with sampling noise on top



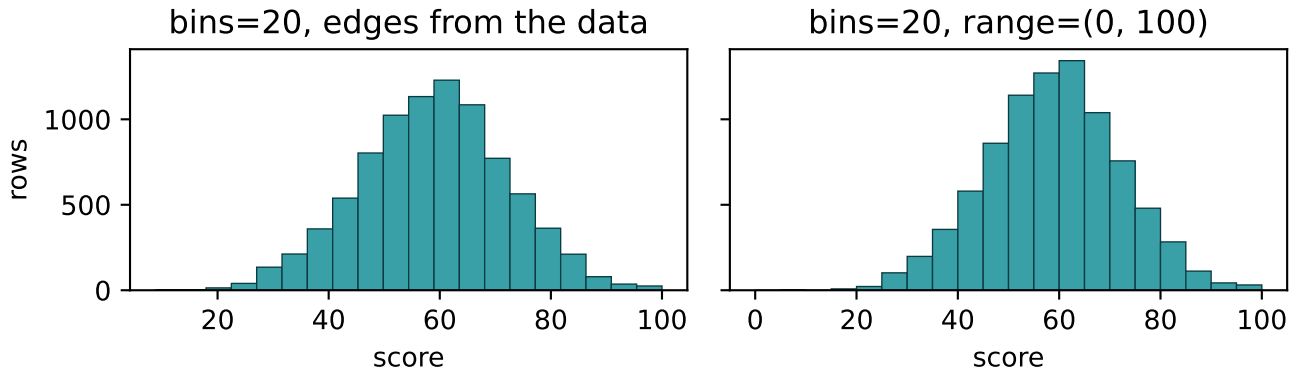
The same 8627 scores over 0 to 100 with 5, 20 and 60 bins. Each panel has its own y axis.

NumPy can choose the count: `bins="auto"` takes the smaller bin width of two standard rules, Sturges' and Freedman-Diaconis', and gives 54 bins for these scores. A feature that appears at one bin count and not at the next is a property of the binning rather than of the data.

RANGE

Without `range`, NumPy uses the smallest and largest value present. That makes the bin edges depend on the data rather than on the quantity, and the edges stop being round numbers.

```
np.histogram(s, bins=20)[1][:3]          # -> [ 8.8  13.36  17.92]   from the data
np.histogram(s, bins=20, range=(0, 100))[1][:3]   # -> [ 0.  5. 10.]   from the definition
```



Left, the edges start at the smallest score, 8.8. Right, the edges start at 0, and the empty bin below 5 shows.

Score is defined on 0 to 100, so 0 to 100 is the range the quantity defines, even though no customer scored below 8.8. That has two effects on what the figure can be compared with:

- **Two histograms become comparable.** Binned over their own observed ranges, the scores of two regions get different bin edges and cannot be compared bin by bin.
- **Empty bins show where there are no values.** The first bin is empty because no customer scores below 5; an axis starting at 8.8 leaves that bin out.

Where the quantity has no defined range, as `income` does not, the observed range is the only option, either whole or with the tail cut off.

EMPTY BINS IN THE TAIL

A bell-shaped column binned finely enough has empty bins in its tails, where no value happened to fall. They are part of the data at that bin width.

10

Distribution shapes

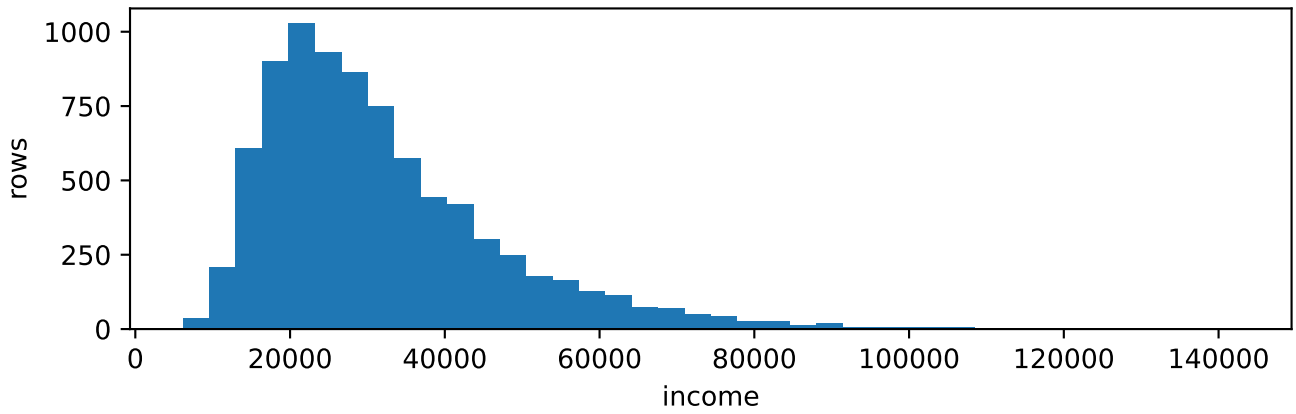
Most histograms have one of a few recognisable shapes, and each shape suggests which statistics describe the column.

SHAPE	APPEARANCE	CONSEQUENCE
Symmetric, one peak	Mean and median nearly equal	The mean and the standard deviation describe it
Right-skewed	Long tail to the right, mean above median	The median describes it better than the mean; a log scale makes the tail readable
Two peaks	Two separate humps	Two populations are mixed, and another column may separate them
A spike at one value	One bin far above its neighbours	Usually a default, a cap, or missing values written as a number
Values at the edge	The first or last bin is tall	The quantity is clipped, so the extremes are not real extremes

The register's `score` is the first shape: one rounded peak near 59, thin on both sides, mean 58.85 against median 59.10. Its `income` is the second: mean 31838 against median 28210, a long right tail, and a histogram in which most of the width is nearly empty.

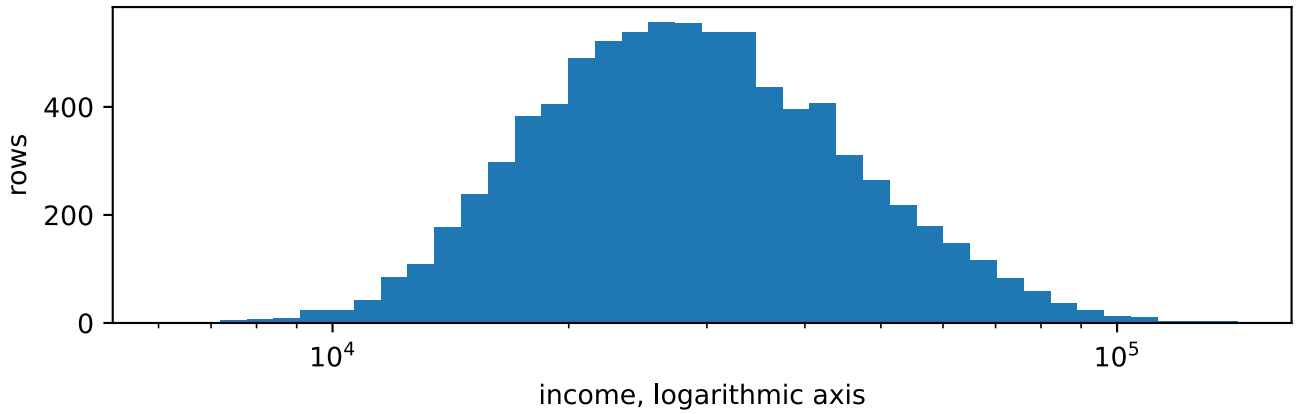
LOG SCALE

```
plt.figure(figsize=(7, 2.4))
plt.hist(inc, bins=40)                  # 40 bins of equal width, about 3410 each
plt.xlabel("income")
plt.ylabel("rows")
plt.tight_layout()
plt.show()
```



On an ordinary axis the histogram is a peak near 20000, where the tallest bar holds 1027 rows, followed by a tail. The right half of the figure, 20 of the 40 bars, holds 167 of the 8237 incomes, 2 per cent.

```
edges = np.geomspace(inc.min(), inc.max(), 41)   # 41 edges, each 1.082 times the last
plt.figure(figsize=(7, 2.4))
plt.hist(inc, bins=edges)
plt.xscale("log")
plt.xlabel("income, logarithmic axis")
plt.ylabel("rows")
plt.tight_layout()
plt.show()
```



A logarithmic axis spaces the values by ratio instead of by difference: the distance from 10000 to 20000 is the same as the distance from 50000 to 100000. Income's largest value is 23 times its smallest, and on this axis the whole range is readable. `np.geomspace(start, stop, n)` returns `n` values with a constant ratio between neighbours, the logarithmic counterpart of `np.linspace`, so the 40 bins come out equally wide on the logarithmic axis. The equal-width bins of the first figure would be wide on the left of it and narrow on the right. The axis label states that the axis is logarithmic, because distances on it are ratios rather than differences.

SPIKES AT ONE VALUE

A tall bar at exactly zero in a column of amounts, or at exactly the same value in several columns, is often missing data filled in with a fixed value before the file was written.

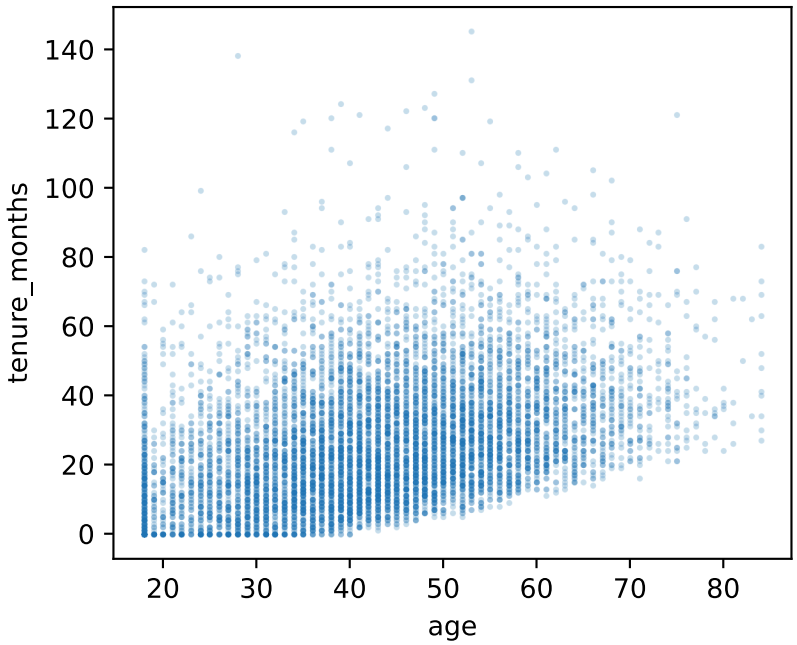
11

Scatter plots

A scatter plot puts one point per row at the coordinates of two columns. Unlike a histogram, it shows the individual rows rather than a summary of them.

```
k = ~np.isnan(age) & ~np.isnan(tenure)   # pairwise complete: both present
k.sum()                 # -> 8436 of 9000 rows have both

plt.figure(figsize=(4.4, 3.6))
plt.scatter(age[k], tenure[k], s=5, alpha=0.25, linewidths=0)
plt.xlabel("age")
plt.ylabel("tenure_months")
plt.tight_layout()
plt.show()
```

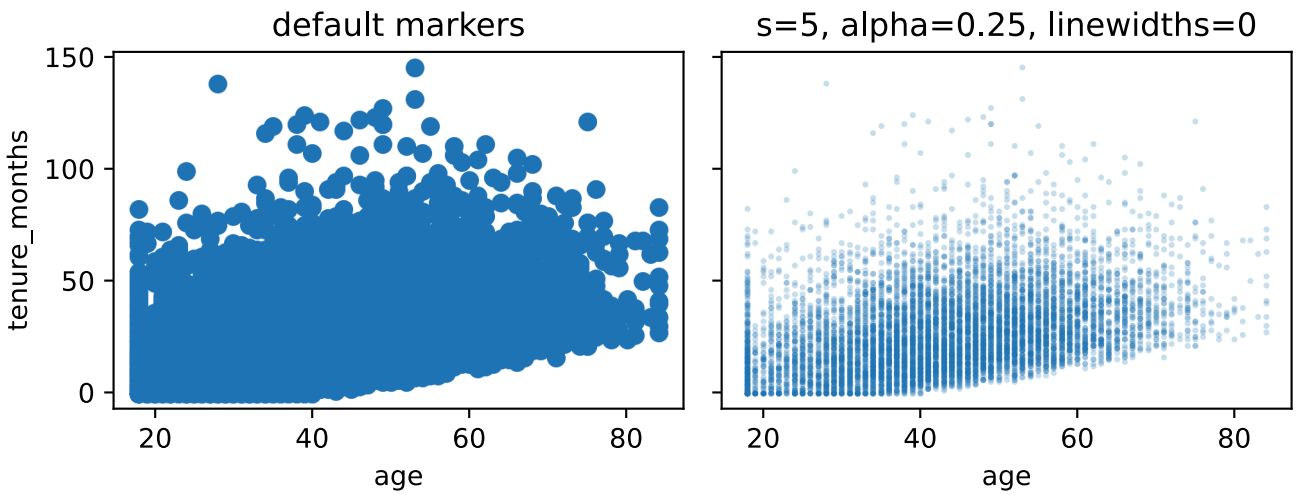


Both arrays passed to `scatter` must be the same length, and the mask has to be the same mask for both, or the points pair up the wrong rows. One combined mask is therefore applied to both columns.

OVERPLOTTING

8436 points in a small figure overlap into a solid area in which dense and sparse regions look the same. Three arguments reduce the overlap:

- `s=5` shrinks the markers, so fewer of them touch.
- `alpha=0.25` makes them translucent, so four overlapping points are visibly darker than one. The dense core of the cloud becomes readable.
- `linewidths=0` removes the marker outline, which at this size makes up most of the marker.



The same 8436 points with the default markers, left, and with the three arguments above, right.

For many more points, a joint histogram, which counts the points in each cell of a grid, shows the density without drawing every point.

DIRECTION, SHAPE, SPREAD, EXCEPTIONS

A scatter plot is read for four things, in this order: the **direction** (whether y rises with x), the **shape** (a straight band, a curve, a fan), the **spread** (tight or diffuse), and the **exceptions** (points far from the rest). Age against tenure in the register rises: the median tenure is 12 months for customers aged 18 to 27 and 38 months for those aged 78 and over. The band is wide and straight. Along its lower edge 317 customers sit at exactly 0 months, 294 of them younger than 35, and nine points lie far above it, at more than 120 months.

12

Correlation

The correlation coefficient measures how closely two columns follow a straight line. `np.corrcoef` computes the Pearson product-moment correlation coefficient, which is what "correlation" means unless another is named.

```
k = ~np.isnan(age) & ~np.isnan(tenure)
np.corrcoef(age[k], tenure[k])          # -> (2, 2) matrix, not one number
np.corrcoef(age[k], tenure[k])[0, 1]    # -> +0.391   the coefficient
```

`np.corrcoef` returns the matrix of every pair, so two inputs give a 2 by 2 with ones on the diagonal, and the coefficient of the pair is off the diagonal. The coefficient runs from -1 to +1: +1 is a perfectly rising straight line, -1 a perfectly falling one, 0 no straight-line relationship at all.

The six numeric pairs in the register, each over the rows where both columns are present:

PAIR	R	ROWS	READING
`age`, `tenure_months`	+0.391	8436	Older customers have longer tenure
`age`, `income`	+0.322	7720	Income rises with age
`income`, `score`	+0.227	7895	A weak rise
`income`, `tenure_months`	+0.123	8237	Very weak
`tenure_months`, `score`	+0.103	8627	Very weak
`age`, `score`	+0.058	8089	None

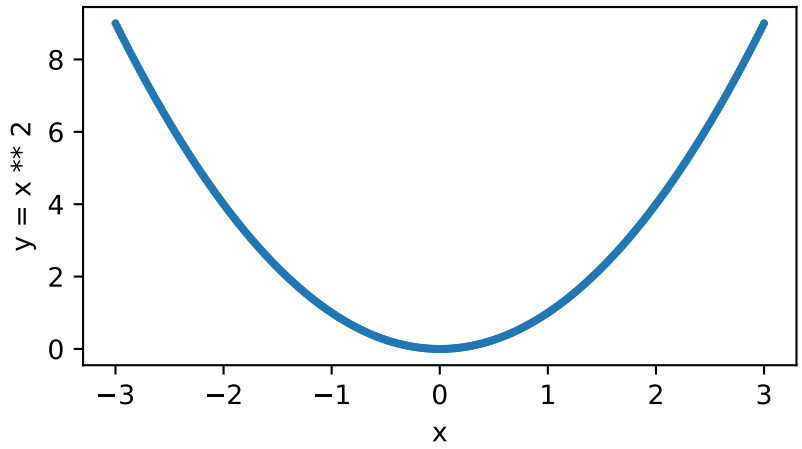
Each pair has its own row count, because each pair loses a different set of rows to missing values. Computed over only the rows complete in all four columns, the six coefficients would describe a different population, as listwise deletion does for a mean.

NONLINEAR RELATIONSHIPS

An exact relationship can still give a correlation of zero:

```
x = np.linspace(-3, 3, 400)
y = x ** 2                              # y is completely determined by x
np.corrcoef(x, y)[0, 1]                 # -> -0.000000
```

Every value of y is fixed by x, and the coefficient is zero, because as x rises y first falls and then rises and the two halves cancel. A correlation near zero rules out a straight-line relationship only; a scatter plot shows whether another one exists.



The 400 points of y = x ** 2. Correlation 0.000.

CORRELATION AND CAUSATION

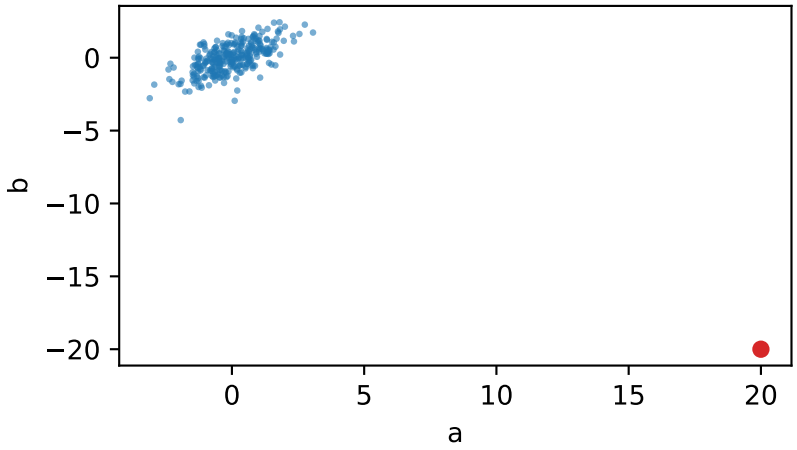
A correlation coefficient does not indicate which column causes which, or whether either does. Two columns both driven by a third correlate with each other, and nothing in the number marks that case.

INFLUENCE OF A SINGLE POINT

```
rng = np.random.default_rng(0)          # a random generator with a fixed seed
a = rng.normal(0, 1, 300)               # 300 values, mean 0, standard deviation 1
b = 0.6 * a + rng.normal(0, 0.8, 300)   # 0.6 * a plus noise
np.corrcoef(a, b)[0, 1]                 # -> +0.637

a2 = np.append(a, 20.0)                 # one point, far out, in the opposite corner
b2 = np.append(b, -20.0)
np.corrcoef(a2, b2)[0, 1]               # -> -0.286   sign reversed by one row in 301
```

`np.random.default_rng(0)` creates a random number generator seeded with 0, so the same values come out on every run, and `rng.normal(mean, std, n)` draws `n` values from a normal distribution. The coefficient is built from products of distances from the mean, so a point far from both means dominates the sum. The scatter plot shows that point.



The 300 points of a and b, correlation +0.637, and the added point at (20, -20) in red, with which it is -0.286.

13

Joint histograms

A histogram of two columns at once, called a joint histogram, or in NumPy's own wording a bi-dimensional histogram. The plane is divided into a grid of cells and each cell counts the rows that fall inside it.

```
k = ~np.isnan(score) & ~np.isnan(tenure)
J, xe, ye = np.histogram2d(score[k], tenure[k], bins=[5, 5],
                           range=[[0, 100], [0, 180]])

J.shape                 # -> (5, 5)   counts, as float64
J.sum()                 # -> 8627.0   every pair is in a cell
xe                      # -> [  0.  20.  40.  60.  80. 100.]   score edges
ye                      # -> [  0.  36.  72. 108. 144. 180.]   tenure edges
```

`bins` and `range` take one entry per axis. The counts come back as **float64**, so a count prints as `2865.0`. The first argument indexes the *first* axis of the result, so `J[i, j]` is the number of rows whose score is in band `i` and whose tenure is in band `j`.

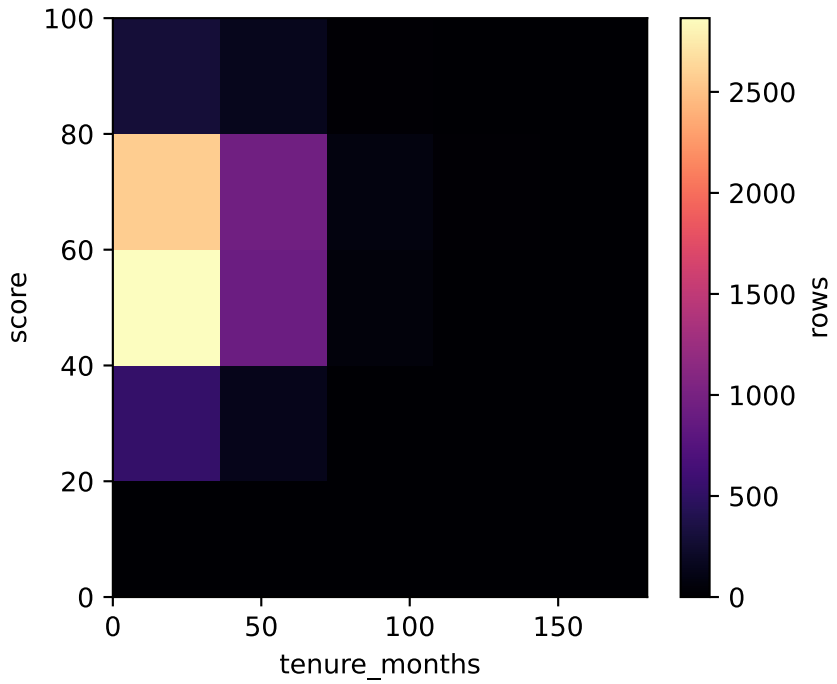
```
J.astype(int)
# [[   7    2    0    0    0]       score 0-20,   across the five tenure bands
#  [ 530  139    8    1    0]       score 20-40
#  [2865  914   66    6    1]       score 40-60
#  [2570  948   89   12    0]       score 60-80
#  [ 297  152   19    1    0]]      score 80-100

J[0]                    # -> the lowest score band, across tenure
J[:, 0]                 # -> the lowest tenure band, across score
int((J == 0).sum())     # -> 6 of 25 cells are empty
```

Six of the twenty-five cells are empty. No customer scoring below 20 has a tenure of 72 months or more, and one customer, with a score between 40 and 60, has a tenure of 144 months or more.

PCOLORMESH AND THE AXIS ORDER

```
plt.figure(figsize=(4.6, 3.8))
plt.pcolormesh(ye, xe, J, cmap="magma")   # edges for x, then y, then the counts
plt.colorbar(label="rows")
plt.xlabel("tenure_months")
plt.ylabel("score")
plt.tight_layout()
plt.show()
```



`pcolormesh(X, Y, C)` treats the first index of `C` as the **y** direction and the second as x, while `histogram2d` put the first argument on the first index. Passing the tenure edges as X and the score edges as Y therefore lines up, because score is `J`'s first index. Passed the other way round, the figure comes out transposed with no error, so both axes are labelled and one cell is checked against the printed matrix.

With many rows, a joint histogram replaces the scatter plot: it counts the rows in each cell instead of drawing each row, so the density stays readable where points would overlap.

14

Comparing groups

A label splits the rows into classes, and any count, summary or figure can be computed once per class. Here the label is `churn`, and the two classes are the customers who stayed and the customers who left.

CLASS BALANCE

```
churn = np.array([int(r["churn"]) for r in records])   # 1 for a customer who left

np.bincount(churn)                  # -> [5716 3284]   rows per class
np.bincount(churn) / len(churn)     # -> [0.6351 0.3649]
```

`np.bincount` counts how often each of 0, 1, 2 and so on occurs, so on a 0/1 label it returns the size of each class. 63.5 per cent of the customers stayed, so a model that predicts "stayed" for every row, whatever its features, is right 63.5 per cent of the time. The share of the largest class is the baseline an accuracy on this label is compared with. A label whose classes differ a lot in size is called **imbalanced**.

THE MEAN OF A 0/1 COLUMN

```
churn.mean()                          # -> 0.3649   the share of ones

for name in ["basic", "standard", "premium", "legacy"]:
    k = plan == name
    print(f"{name:<9} n={k.sum():>5}  churn rate {churn[k].mean():.3f}")

# basic     n= 3103  churn rate 0.334
# standard  n= 2819  churn rate 0.419
# premium   n= 1828  churn rate 0.181
# legacy    n= 1250  churn rate 0.589
```

The sum of a 0/1 column is its number of ones, so its mean is the share of ones, and a grouped mean of the label is the rate of the label within each group. Customers on the legacy plan left at three times the rate of customers on the premium plan, so `plan` is a column a classifier can use.

MEANS PER CLASS

```
for name, v in [("age", age), ("income", income), ("tenure", tenure), ("score", score)]:
    stayed = np.nanmean(v[churn == 0])
    left = np.nanmean(v[churn == 1])
    print(f"{name:<7} stayed {stayed:9.2f}   left {left:9.2f}")

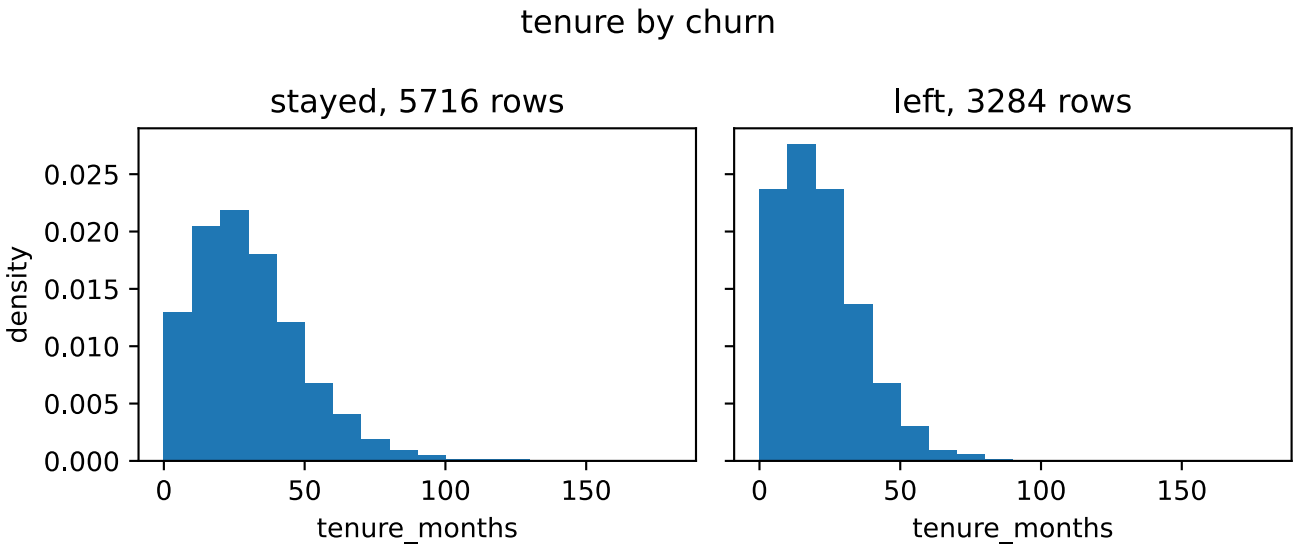
# age     stayed     44.07   left     41.46
# income  stayed  33521.97   left  28923.53
# tenure  stayed     29.96   left     21.10
# score   stayed     62.02   left     53.31
```

The customers who left had been customers for 9 months less on average, and scored 9 points lower. A difference between two means measures how far apart the centres of the two classes are, and not how far the two distributions overlap, which the histograms of the two classes show.

PLT.SUBPLOTS

```
fig, axes = plt.subplots(1, 2, figsize=(7, 3), sharex=True, sharey=True)

for ax, label, name in zip(axes, [0, 1], ["stayed", "left"]):
    k = churn == label                  # tenure has no gaps, so no NaN mask
    ax.hist(tenure[k], bins=18, range=(0, 180), density=True)
    ax.set_title(f"{name}, {k.sum()} rows")
    ax.set_xlabel("tenure_months")
axes[0].set_ylabel("density")
fig.suptitle("tenure by churn")
fig.tight_layout()
plt.show()
```



`plt.subplots(nrows, ncols)` creates a figure divided into a grid of plotting areas and returns two objects: the `Figure`, and the plotting areas, which matplotlib calls `Axes`. One row of two gives a NumPy array of two `Axes`, a grid gives a two-dimensional array indexed as `axes[row, col]`, and a single plot gives one `Axes` rather than an array. `fig.suptitle` puts one title above all the panels.

Each `Axes` has the plotting functions as methods, such as `ax.hist` and `ax.scatter`, and the functions that set a property take a `set_` prefix: `plt.xlabel` becomes `ax.set_xlabel`, and `plt.title` becomes `ax.set_title`. Matplotlib calls this the explicit Axes interface. Calls such as `plt.hist` and `plt.xlabel` use the implicit pyplot interface, which draws on whichever Axes was created last; with several panels in one figure, the explicit form names the panel every call draws on.

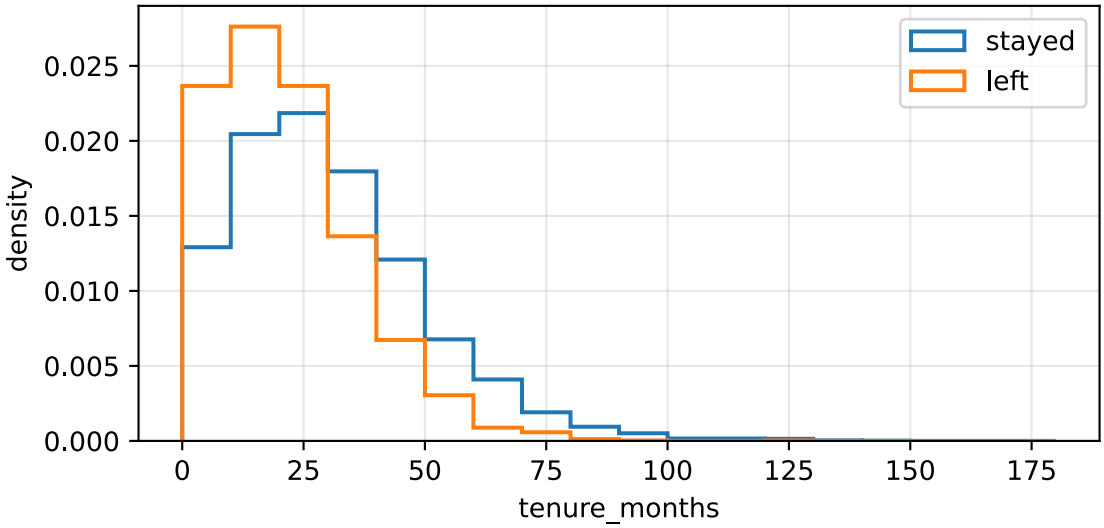
`sharex=True` and `sharey=True` give every panel the same axis limits. Without them each panel is scaled to its own data, the tallest bar reaches the top of every panel, and two different distributions can look alike.

DENSITY=TRUE FOR GROUPS OF DIFFERENT SIZE

The two classes have 5716 and 3284 rows, so raw counts mix the size of each class into its shape. In the bin from 10 to 20 months the counts are 1169 for the customers who stayed and 907 for those who left. As a share of each class the order reverses: 20.5 per cent of the customers who stayed fall in that bin, against 27.6 per cent of those who left. `density=True` divides each histogram by its own number of rows and by the bin width, so the bars of the two classes can be compared directly.

LEGENDS AND GRIDS

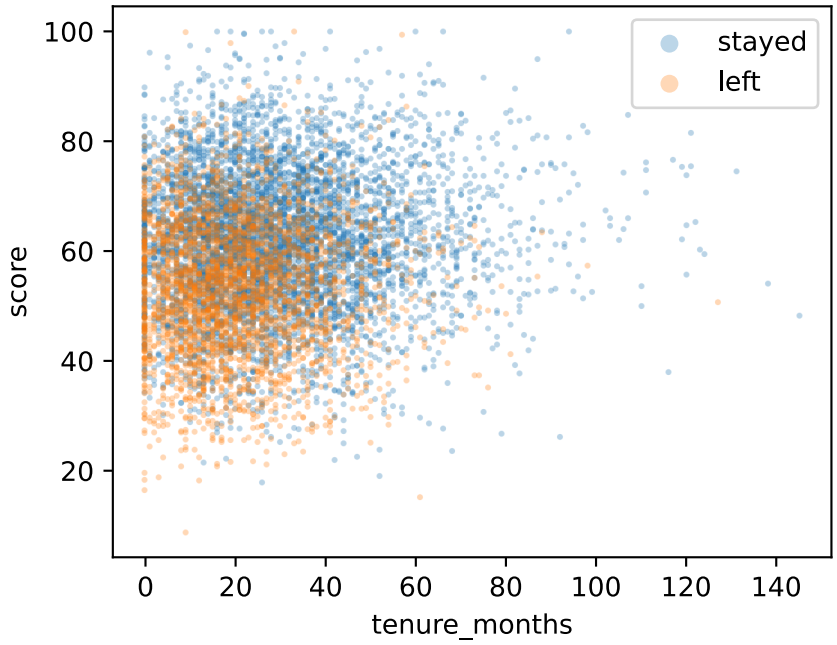
```
fig, ax = plt.subplots(figsize=(6, 3))
for label, name in [(0, "stayed"), (1, "left")]:
    ax.hist(tenure[churn == label], bins=18, range=(0, 180), density=True,
            histtype="step", linewidth=1.5, label=name)
ax.set_xlabel("tenure_months")
ax.set_ylabel("density")
ax.legend()
ax.grid(alpha=0.3)
fig.tight_layout()
plt.show()
```



Both classes on one `Axes`. `histtype="step"` draws each histogram as an outline, so neither covers the other. `label=` names each call, and `ax.legend()` draws a key from those names. `ax.grid(alpha=0.3)` adds faint grid lines to read the heights against.

SCATTER PLOTS BY CLASS

```
fig, ax = plt.subplots(figsize=(4.6, 3.6))
for label, name in [(0, "stayed"), (1, "left")]:
    k = (churn == label) & ~np.isnan(score)
    ax.scatter(tenure[k], score[k], s=5, alpha=0.3, linewidths=0, label=name)
ax.set_xlabel("tenure_months")
ax.set_ylabel("score")
ax.legend(markerscale=3)
fig.tight_layout()
plt.show()
```



One `scatter` call per class, each with its own colour and label, shows two features and the label in one figure. `markerscale=3` enlarges the markers in the key, which at `s=5` are hard to see. The customers who left lie towards short tenure and low score, and the two clouds overlap over most of the plane. To colour the points by

a numeric column instead, the column goes to the `c` argument and a colour map to `cmap`, as in `sc = ax.scatter(x, y, c=v, cmap="viridis")`, and `fig.colorbar(sc, ax=ax)` adds the colour scale.

15

Encoding a table for a neural network

A neural network computes weighted sums of its inputs: each input is multiplied by a weight, the products are added, and training adjusts every weight by a step proportional to the input it multiplies. That requires three things of the input: a number in every cell, no gaps, and columns on comparable scales. The register has three text columns, gaps in three numeric columns, and an income in tens of thousands next to a score out of 100. The steps below turn it into a float32 matrix that meets all three.

REQUIREMENTS ON THE INPUT

- **Every input is a number.** Each input is multiplied by a weight, and a string cannot be.
- **No input is missing.** One NaN makes every weighted sum it enters NaN, and during training it spreads to every weight.
- **The inputs are on comparable scales.** A column in tens of thousands next to one in tens needs weights hundreds of times apart, and its weight is moved by training steps hundreds of times larger, so no single step size suits both.

TEXT COLUMNS

```
region[:3]              # -> ['North' 'Eastfield' 'Midland']
region[:3] * 0.5
# UFuncTypeError: ufunc 'multiply' did not contain a loop with signature matching types
# (dtype('<U9'), dtype('float64')) -> None
```

NumPy has no multiplication of a string by a number, so a text column cannot enter a weighted sum. It has to become one or more numeric columns first.

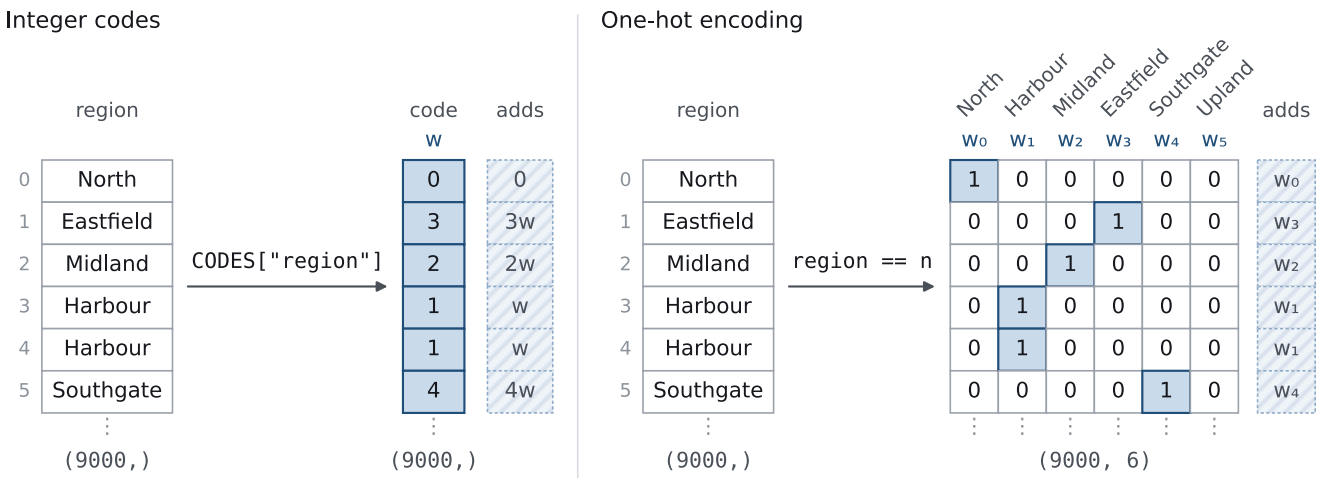
INTEGER CODES

The integer codes replace each region with its position in the list: North 0, Harbour 1, Midland 2, Eastfield 3, Southgate 4, Upland 5. As an input, that column is multiplied by a single weight `w`, so the six regions contribute 0, `w`, `2w`, `3w`, `4w` and `5w`: equal steps, in the order the list happens to have. The mean scores by region, taken in code order, are 51.3, 60.8, 58.6, 65.2, 55.7 and 67.5, which rise and fall alternately, and no value of `w` produces that pattern from the codes.

ONE-HOT ENCODING

```
names = CATEGORIES["region"]
onehot = np.stack([(region == n).astype(np.float32) for n in names], axis=1)

onehot.shape            # -> (9000, 6)   one column per region
onehot[:3]
# [[1. 0. 0. 0. 0. 0.]      North
#  [0. 0. 0. 1. 0. 0.]      Eastfield
#  [0. 0. 1. 0. 0. 0.]]     Midland
onehot.sum(axis=1)      # -> [1. 1. 1. ... 1. 1. 1.]   exactly one 1 in every row
onehot.sum(axis=0)      # -> [1928. 1634. 1798. 1262. 1417.  961.]   rows per region
```



The region of rows 0 to 5 as one column of integer codes, and as six one-hot columns in the order of CATEGORIES["region"]. The hatched columns are what each row adds to a weighted sum: a multiple of the single weight w from the code, and the weight of its own column from the one-hot columns.

Each region becomes a column of its own, holding 1 in the rows of that region and 0 everywhere else. `np.stack` joins the six `(9000,)` columns along a new axis, and `axis=1` makes that the second axis, so the result has one row per customer. Every column gets its own weight, so every region contributes an amount of its own, independent of the other five. The columns follow the order of `CATEGORIES["region"]`, which has to be the same every time the encoding is built, or the fifth column would mean Southgate in one matrix and another region in the next.

DROPPING INCOMPLETE ROWS

Listwise deletion gives a complete matrix by keeping only the 7403 rows with no gap. It discards 1597 rows, 18 per cent of the table, with every value that was present in them. It also does not help once the network is in use: a new customer with no recorded income still needs a number in that cell before the network can produce a prediction.

IMPUTATION WITH THE MEDIAN

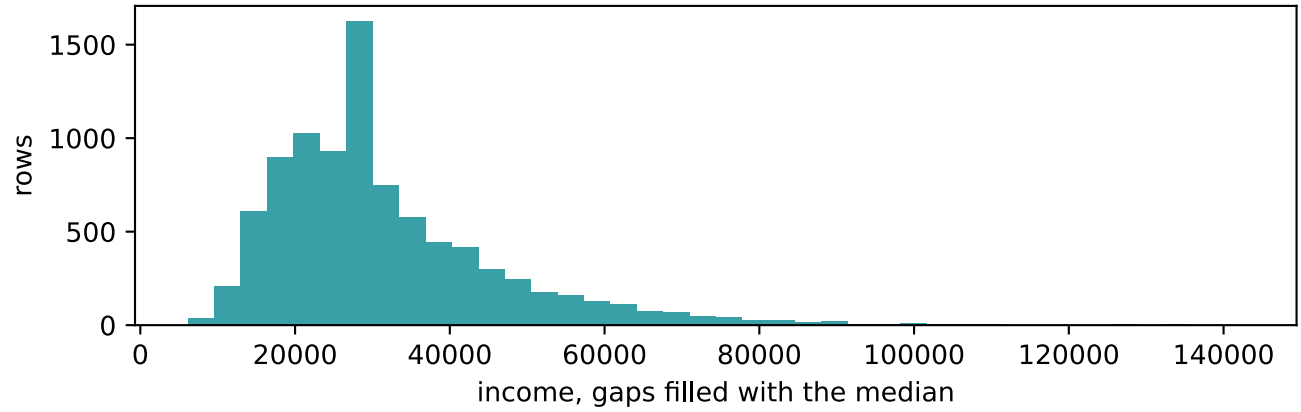
```
num = np.stack([column(c) for c in NUMERIC], axis=1)     # (9000, 4) float64, NaN in the gaps
med = np.nanmedian(num, axis=0)                 # -> [   43.   28210.26    24.      59.1 ]
filled = np.where(np.isnan(num), med, num)      # (9000, 4) v (4,) -> (9000, 4)

filled[15]              # -> [   52.   28210.26    23.      65.8 ]   the income was a gap
np.isnan(filled).any()  # -> False
```

Imputation replaces each gap with a value computed from the column. The median is the usual choice for a skewed column such as income, because a long tail moves the mean and not the median.

`np.where(condition, a, b)` takes `a` where the condition holds and `b` elsewhere; `med` has shape `(4,)` and broadcasts across the rows, so each column is filled with its own median.

Filling places 763 incomes exactly at the median, so the filled column is narrower than the recorded one: its standard deviation drops from 15492 to 14855. The filled column also contains nothing that distinguishes a filled value from a recorded one.



Income after median imputation. The bar holding the median, 26609 to 30021, grows from 863 to 1626 rows with the 763 filled values; every other bar is unchanged.

INDICATOR COLUMNS

```
flags = np.isnan(num[:, [0, 1, 3]]).astype(np.float32)   # age, income and score have gaps

flags.shape             # -> (9000, 3)
flags[15]               # -> [0. 1. 0.]   row 15's income was filled
flags.sum(axis=0)       # -> [564. 763. 373.]
```

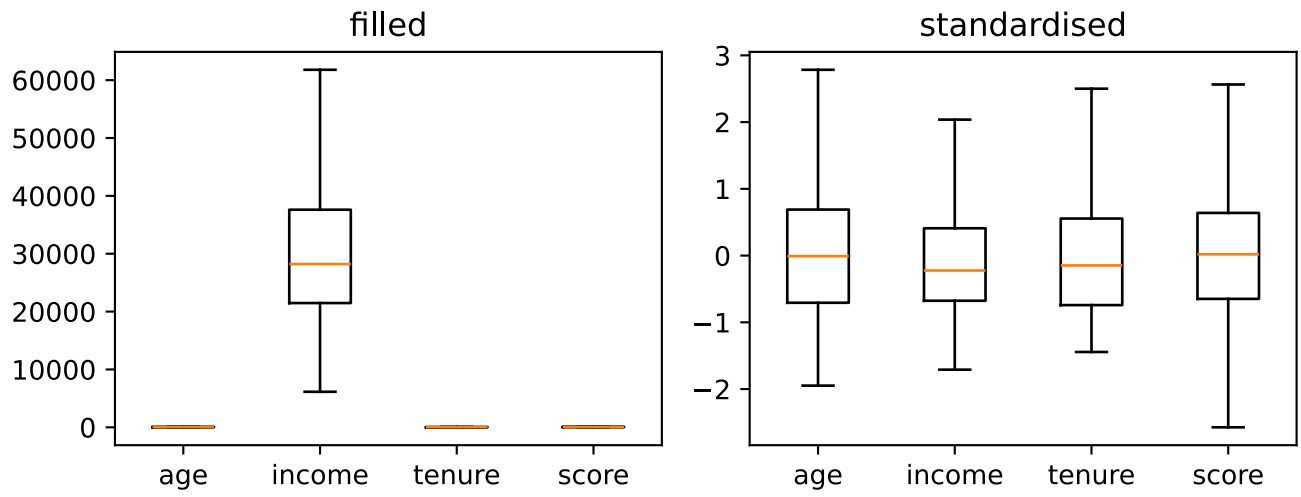
One extra 0/1 column for each column that had gaps, holding 1 where the value was filled in. Whether a value is missing can itself be related to the label, for instance when customers who do not state their income behave differently from those who do, and that information is lost when only the filled value is kept. Filling the gaps and adding these columns is called the **missing-indicator method**.

STANDARDISATION

```
filled.std(axis=0)      # -> [   12.89 14854.88    18.51    12.89]

z = (filled - filled.mean(axis=0)) / filled.std(axis=0)   # (9000, 4) v (4,) -> (9000, 4)
z.std(axis=0)           # -> [1. 1. 1. 1.]
z[15]                   # -> [ 0.69 -0.22 -0.2   0.54]
```

The standard deviation of income is about 1150 times that of score. With equal weights, a typical difference in income moves a weighted sum about 1150 times as far as a typical difference in score, so for the two to have a similar influence the weights would have to differ by the same factor; and since training moves a weight by a step proportional to its input, their steps would differ by that factor too. **Standardising** subtracts each column's mean and divides by its standard deviation. Afterwards every column has mean 0 and standard deviation 1, and a value reads as a number of standard deviations from the mean: the score of row 15, 65.8, becomes 0.54.

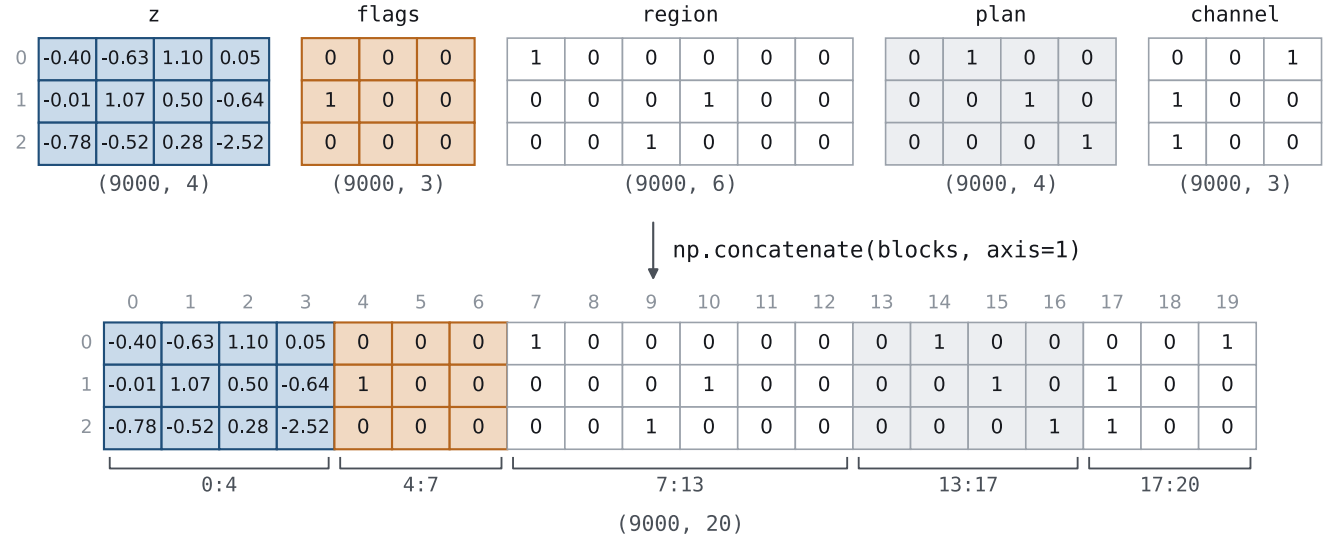


The four filled columns before and after standardising, as box plots: each box spans the middle half of a column, the line inside it is the median, and the whiskers reach at most 1.5 box lengths beyond it. Before standardising, income's scale compresses the other three boxes; after it, all four share one axis.

THE FEATURE MATRIX

```
blocks = [z, flags]
for c in CATEGORICAL:
    v = np.array([r[c] for r in records])
    blocks.append(np.stack([(v == name).astype(np.float32) for name in CATEGORIES[c]],
                           axis=1))
X = np.concatenate(blocks, axis=1).astype(np.float32)

X.shape                 # -> (9000, 20)   4 standardised + 3 indicators + 6 + 4 + 3 one-hot
np.isnan(X).any()       # -> False
X.dtype                 # -> dtype('float32')
```



np.concatenate joins the blocks along axis 1 in list order, so each block becomes a range of columns of X: z is X[:, 0:4] and the region columns X[:, 7:13]. Rows 0 to 2 are drawn, with the standardised values rounded to two decimals.

`np.concatenate` joins arrays along an axis they already have, here the columns. Every cell of `X` is a number, none is missing, and the numeric columns share one scale. The final `astype` gives float32, the dtype of a network's weights.

The medians, means and standard deviations above come from all 9000 rows. Once the rows are split into a set to train on and a set to test on, these statistics are computed on the training rows only and reused unchanged

on the test rows, so that nothing about the test rows reaches the model before it is tested.

16

Loop forms and array equivalents

The counting, summarising, binning and encoding loops, written as array expressions over a column `v`, two or three columns `x`, `y` and `z`, a 0/1 column `label`, or a matrix `m` of numeric columns. The summaries skip missing values rather than propagating them.

WHAT THE LOOP DOES	AS AN ARRAY EXPRESSION
Rows in one category	`(plan == "premium").sum()`
Rows in every category, in a chosen order	`[(plan == n).sum() for n in CATEGORIES["plan"]]`
Distinct categories and their counts	`np.unique(plan, return_counts=True)`
How many values are missing	`np.isnan(v).sum()`
How many are present	`(~np.isnan(v)).sum()`
Mean, skipping the gaps	`np.nanmean(v)`
Median and spread, skipping the gaps	`np.nanmedian(v)`, `np.nanstd(v)`
Extremes, skipping the gaps	`np.nanmin(v)`, `np.nanmax(v)`
Quartiles	`np.quantile(v[~np.isnan(v)], [.25, .5, .75])`
Mean within one group	`v[(cat == name) & ~np.isnan(v)].mean()`
Count values in one interval	`((v >= lo) & (v < hi)).sum()`
Count values in every interval	`np.histogram(v, bins=20, range=(0, 100))`
The bin a value falls into	`np.digitize(v, edges)`
Counts scaled to unit area	`np.histogram(v, bins=20, density=True)`
Rows where two columns are both present	`~np.isnan(x) & ~np.isnan(y)`
Count rows in a cell of a grid	`np.histogram2d(x, y, bins=[5, 5])`
Linear agreement of two columns	`np.corrcoef(x[k], y[k])[0, 1]`
All pairwise correlations at once	`np.corrcoef(np.stack([x, y, z]))`
Rows in each class of a 0/1 label	`np.bincount(label)`
Share of ones in a 0/1 column	`label.mean()`
Mean of a column within each class	`[np.nanmean(v[label == c]) for c in (0, 1)]`
One 0/1 column per category	`np.stack([(v == n).astype(np.float32) for n in names], axis=1)`
Each column's gaps filled with its median	`np.where(np.isnan(m), np.nanmedian(m, axis=0), m)`
Where the gaps were, as 0/1 columns	`np.isnan(m).astype(np.float32)`
Every column standardised, after filling	`(m - m.mean(axis=0)) / m.std(axis=0)`

FURTHER READING

- The csv module, for `DictReader`, dialects and quoting.
- Statistics routines, which lists the NaN-ignoring function beside each ordinary one.
- isnan and unique.

- pyplot.bar, for the arguments the example above does not use.
- Imputation of missing values, scikit-learn's guide to filling strategies and to marking the filled values.
- histogram and histogram2d, including the `weights` and `density` arguments.
- corrcoef, and cov underneath it.
- pyplot.hist, scatter and pcolormesh.
- pyplot.subplots, and Matplotlib Application Interfaces on the explicit and the implicit interface.